

Project #62: Time-series ML Model Selection Framework¹

Project Scope, Objectives, and Literature Review

Trisan Ethan Q Huynh, partnered with Scott Mouat; supervised by Andreas Kempa-Liehr

Project Scope

Engineering and scientific fields increasingly rely on vast amounts of data collected over time, known as time-series data. This data typically exhibits complex dynamics, non-stationarity, and high dimensionality, meaning that a machine learning (ML) centred analysis of this data is particularly challenging [REF].

To process this data, a machine learning workflow requires several intricate phases: data windowing, feature extraction, feature selection, model fitting, and model evaluation. Models that are deployed in a decision-making or business role may typically also require long-run model performance evaluation and subsequent retraining.

The feature extraction phase is particularly complex, as a decision must be made between:

- Manual, subjective feature engineering, or;
- Automated feature extraction libraries, such as `tsfresh`, `catch22`, `TSFEL`, etc.

This leads into the core problem: the complexity of these existing layered workflows makes it very hard to identify improvements that can be made to time-series ML pipelines. There is a lack of a unified, standardised workflow to test how changes made to these pipelines – for example, swapping one feature extraction tool for another – impacts the performance across various datasets.

As such, the scope of this project is to develop and implement a dedicated high-performance computing framework. This framework will extend existing massively parallel machine learning systems to handle the specific challenges posed by time-series data. Ultimately, it will allow users to comprehensively verify and cross-validate problem-specific pipelines, compare variations, and generate tests to identify statistically significant improvements that can be made.

¹ Could also go by Time-series Evaluation for Model Performance Optimisation (TEMPO).

Project Objectives

Now that the scope of the project has been defined, it is now possible to identify the key objectives of the project.

These are:

- Review literature: analyse the approaches taken by existing feature extraction pipelines, such as `tsfresh`, `catch22`, and `TSFEL`, among others. Also, research the current state of affairs regarding massively parallel ML workflows.
- Identify workflow improvements: identify current limitations in automating and evaluating time-series machine learning pipelines.
- Design framework architecture: draft an extension for existing massively parallel ML systems, with the intention of handling time-series specific challenges.
- Construct extension: develop the extension, including feature extraction and selection, model fitting and selection, as well as the implementation of cross validation.
- Test: benchmark the performance of these problem-specific pipelines against a diverse range of industry-standard datasets.
- Vary components: procedurally swap components (e.g. feature extactor) to measure the impact on performance.
- Statistical analysis: develop tests that allow identification of statistically significant pipeline improvements.

Literature Review

Time-Series Machine Learning

Time-series data, which consists of measurements collected sequentially over time, is becoming ever-present in modern engineering and science applications, such as predictive maintenance and financial forecasting [1]. However, analysing this data is uniquely challenging, as it often exhibits high dimensionality, non-stationarity, and regularly contains complex, non-linear dynamics [1].

Historically, researchers relied on classical econometric models like ARIMA or exponential smoothing [2]. While useful, these models often do not have the ability to adapt to large changes in data distributions and fail to interpret non-linear patterns. This limitation has driven a shift towards machine learning (ML) models, such as decision tree ensembles and neural networks, in attempts to better capture these relationships [2].

Implementing an ML approach for time-series data typically requires an intricate, multi-stage pipeline, involving:

- Windowing and validation: data is often segmented using a “sliding window” approach, where the training sample sequentially moves along the time series to evaluate the model performance along specific data samples.
- Anomaly detection: raw sequences of time-series data often contain noise and outliers that severely reduce model accuracy [2]. Identifying and managing these anomalies prior to model fitting and testing is a critical preprocessing step to ensure that the performance metrics calculated accurately represent the performance of the model [2].
- Feature engineering: typically, the most intensive phase. Given the accessibility of modern-day sensors, time-series data is typically measured en-masse, resulting in a high-dimensional raw data feed. Feeding this directly into an ML model typically incurs the so-called “curse of dimensionality”, in which the algorithm struggles to find meaningful patterns or statistical significance, due to the sheer volume of the feature space causing the data to become increasingly sparse [1]. To prevent this, the data measurements are reduced into a lower-dimensional set of features that describe the full data well, and typically span the temporal, statistical, and spectral domains [3].
- Feature selection: since automated feature extraction commonly generates thousands of candidate features, aggressive feature selection – typically using greedy algorithms or

hypothesis testing – is used to remove redundant features. This reduces computational load and model fitting time, which prevent model overfitting [2].

- Model fitting and tuning: the final features are finally fed into machine learning models. This includes the computationally expensive process of hyperparameter tuning (using techniques such as grid search) to optimise predictive performance [2].

The complexity of combining these phases makes it difficult to systematically identify and test improvements that could be made to time-series workflows.

Automated Feature Extraction Methods

Traditionally, the process of creating “useful” information for the model – known as feature engineering – relied on subjective, manual efforts. In order to standardise this process and prevent the “curse of dimensionality”, researchers now rely on automated feature extraction libraries that map raw time-series data into lower-dimensional, interpretable feature spaces [1], [4].

A good representation for this workflow is `hctsa`, a comprehensive framework that computes over 7,700 features including autocorrelation, entropy, and non-linear dynamics [5], [6]. While comprehensive, the sheer volume of features generated and reliance on proprietary software (MATLAB, in this case) result in high computational costs², making it impractical for many fast, real-world applications.

To address this issue of scale, the FRESH (FeatuRe Extraction based on Scalable Hypothesis tests) algorithm was developed, popularised by the Python package `tsfresh` [4], [8]. `tsfresh` automatically calculates hundreds of features and applies a hypothesis-testing framework – specifically, the Benjamini-Yekutieli procedure – to strictly control the false discovery rate and filter out irrelevant features [8]. However, while the operations are highly parallelisable, recent empirical evaluations suggest that `tsfresh` suffers from high within-set redundancy; over 50% of its features are highly correlated fast Fourier transform (FFT) coefficients [5]. Additionally, it is computationally slower than newer packages, taking approximately 2.5 seconds to process a 1000 sample time series [5].

² And financial! MATLAB goes for ~NZD\$1700-7000/seat/year [7].

Libraries such as `tsfresh` rely on scalable hypothesis tests to filter out irrelevant features [8]. However, the underlying statistical methods that these take advantage of can present a bottleneck for complex data. Real-world, high-frequency time-series data often consists of non-normal distributions and jump variations [9]. In these scenarios, integrating tests that evaluate the entire distribution, such as the Cramer-vom Mises test, provides a much more robust goodness-of-fit statistic compared to maximum-distance tests like the Kolmogorov-Smirnov test, ensuring critical features are not incorrectly discarded during the selection process [9].

Another prominent library is `TSFEL` (Time Series Feature Extraction Library), which extracts over 60 features across temporal, statistical, and spectral domains [3]. A strength of this library is that it evaluates the mathematical time complexity (e.g. $O(n)$, $O(\log n)$) of every feature [3]. This makes it particularly advantageous for resource-constrained deployment environments, such as embedded devices or wearables [3]. However, much like `tsfresh`, `TSFEL` also exhibits feature redundancy, with tests showing that 90% of its feature variance can be captured by just four principal components [5].

In order to address the computational overhead and collinearity issues of massive features sets, researched developed `catch22` [5]. By analysing the performance of thousands of `hctsa` features across 93 different datasets, they whittled the feature space down to just 22 “canonical”, minimally redundant features [10]. Implemented in C, `catch22` focuses on computational speed, processing 10,000 samples in under 0.5 seconds – and scaling approximately linearly beyond that [10]. This sets it apart from the heavy redundancy typically associated with `tsfresh` and `TSFEL`.

The choice of feature extractor dictates the efficiency of the entire ML pipeline. Because of this, model automated ML frameworks, such as `FEDOT-TSC`, attempt to use evolutionary optimisation to automatically evaluate and select between different feature generators to build the optimal workflow on-the-fly for a specific dataset [11].

AutoML, Parallel Computing Frameworks, and Scalable Architectures

In order to manage the number of choices in feature extraction and model selection, researchers are increasingly turning to Automated Machine Learning (AutoML). A prominent example is `FEDOT-TSC`, a data driven framework that uses evolutionary optimisation to automatically construct and tune composite ML pipelines [11]. By automating the feature design (dynamically

selecting between different generators) and model design, AutoML systems like FEDOT-TSC remove subjective human bias and optimise workflows for specific datasets [11].

However, automating these pipelines across large datasets introduces severe computational challenges. To achieve scalability, modern frameworks heavily rely upon high-performance and parallel computing, which prioritise optimised and deterministic algorithms. For instance, the MiniRocket algorithm achieves SOTA classification accuracy while remaining exceptionally fast by transforming time series using a small, fixed set of convolutional kernels [12]. This enables computations up to 75 times faster than its predecessors. For other sequence-matching tasks, such as Dynamic Time Warping, researchers utilise GPU acceleration (via Nvidia's CUDA) to compute massive alignment matrices in parallel, significantly reducing execution times. This is a technique that can be further explored as an extension to existing non-AutoML pipelines, such as `tsfresh`, which heavily relies on FFTs, as previously discussed.

Finally, a critical assumption made in time-series machine learning is that future operational samples will be drawn from the same distribution as the training data. In reality, time-series data is highly susceptible to concept drift, as underlying physical conditions change, such as operating speed or frequency [10], [13]. For example, in the case of predictive maintenance, a machine's operational environment can fundamentally affect baseline sensor readings, degrading the performance of the model [1]. As such, modern pipelines must incorporate a core tenet of ML operations (MLOps): continuous training (CT). This automatically monitors performance metrics and triggers model retraining when data distributions shift, or model performance drops below an acceptable threshold [13].

Current Limitations and Opportunities

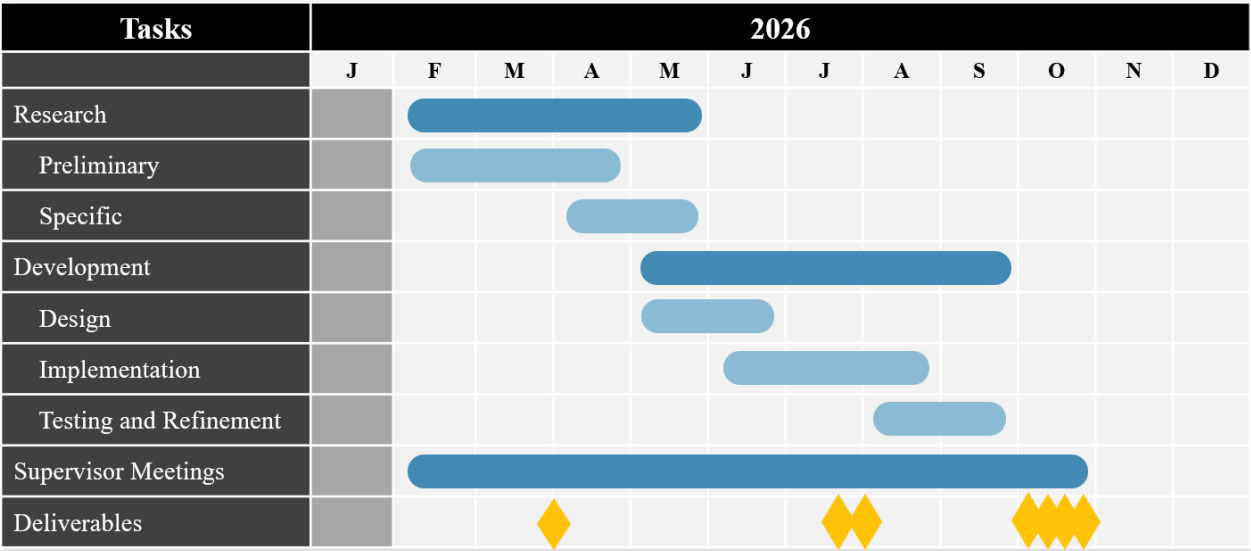
Despite the rapid advancement of automated extraction libraries (such as `tsfresh`, `catch22`, and `TSFEL`) and highly scalable transforms (like MiniRocket), a critical gap remains in the literature. Currently, these sophisticated tools are frequently developed, optimised, and evaluated in isolation from each other [5].

Because time-series workflows require the layered phases as discussed, it is difficult to identify generic, universal improvements to the overall pipeline [2]. Currently, researchers lack a unified and standardised platform to systematically integrate these phases within a single, cross-

validated workflow. For example, if a machine learning engineering wishes to quantify the exact performance impact of swapping `tsfresh` for `catch22`, or substituting a standard classifier for a FFT, they must build an ad-hoc, problem specific testing architecture from scratch [14].

This outlines the objectives and scope of this project, as discussed in the sections above. We must effectively identify generic improvements that can be made – and in order to do this, we must develop a dedicated high-performance computing framework. The work started in the field of AutoML pipelines appear to provide a good point to extend off. By extending these massively parallel machine learning frameworks to accommodate the requirements of time-series data, this project will fill the current gap that exists, enabling scientists and engineers alike to cross-validate and compare various tweaks to a pipeline, and ultimately generate statistically significant statements about pipeline improvements.

Project Schedule



References

- [1] O. O. Aremu, D. Hyland-Wood, and P. R. McAree, 'A machine learning approach to circumventing the curse of dimensionality in discontinuous time series machine data', *Reliab. Eng. Syst. Saf.*, vol. 195, p. 106706, Mar. 2020, doi: 10.1016/j.ress.2019.106706.
- [2] O. Gorodetskaya, Y. Gobareva, and M. Koroteev, 'A Machine Learning Pipeline for Forecasting Time Series in the Banking Sector', *Economies*, vol. 9, no. 4, p. 205, Dec. 2021, doi: 10.3390/economies9040205.
- [3] M. Barandas *et al.*, 'TSFEL: Time Series Feature Extraction Library', *SoftwareX*, vol. 11, p. 100456, Jan. 2020, doi: 10.1016/j.softx.2020.100456.
- [4] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, 'Time Series Feature Extraction on basis of Scalable Hypothesis tests (tsfresh – A Python package)', *Neurocomputing*, vol. 307, pp. 72–77, Sep. 2018, doi: 10.1016/j.neucom.2018.03.067.
- [5] T. Henderson and B. D. Fulcher, 'An Empirical Evaluation of Time-Series Feature Sets', Oct. 21, 2021, *arXiv*: arXiv:2110.10914. doi: 10.48550/arXiv.2110.10914.
- [6] B. D. Fulcher, 'Feature-based time-series analysis', Oct. 02, 2017, *arXiv*: arXiv:1709.08055. doi: 10.48550/arXiv.1709.08055.
- [7] 'Pricing and Licensing'. Accessed: Apr. 02, 2026. [Online]. Available: <https://au.mathworks.com/pricing-licensing.html>
- [8] M. Christ, A. W. Kempa-Liehr, and M. Feindt, 'Distributed and parallel time series feature extraction for industrial big data applications', May 19, 2017, *arXiv*: arXiv:1610.07717. doi: 10.48550/arXiv.1610.07717.
- [9] W. Rojas, R. S. Tsay, and P. A. Morettin, 'A Cramér-von Mises-type test statistic for identifying jump variations in high-frequency time series'.
- [10] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, 'catch22: CANonical Time-series CHaracteristics: Selected through highly comparative time-series analysis', *Data Min. Knowl. Discov.*, vol. 33, no. 6, pp. 1821–1852, Nov. 2019, doi: 10.1007/s10618-019-00647-x.
- [11] I. Revin, V. A. Potemkin, N. R. Balabanov, and N. O. Nikitin, 'Automated machine learning approach for time series classification pipelines using evolutionary optimization', *Knowl.-Based Syst.*, vol. 268, p. 110483, May 2023, doi: 10.1016/j.knosys.2023.110483.
- [12] A. Dempster, D. F. Schmidt, and G. I. Webb, 'MINIROCKET: A Very Fast (Almost) Deterministic Transform for Time Series Classification', in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, Aug. 2021, pp. 248–257. doi: 10.1145/3447548.3467231.
- [13] J. Koskinen, 'Designing a Machine Learning Pipeline with Continuous Training for Time Series Forecasting'.

- [14] Y. Cheng, 'Multivariate Time Series Forecasting through Automated Feature Extraction and Transformer-Based Modeling', vol. 5, no. 5, 2025.